



Aggregatore della programmazione dei cinema dell'Umbria

Team RepCode

Emanuele Ceccariglia · Elio Casciola · Andrea Cestelli · Yonas Burka

Ingegneria del Software ITS Umbria Academy 2025/2026

Il problema

Oggi

Per sapere cosa danno stasera a Perugia devi aprire tre siti diversi, ognuno fatto a modo suo.

PostModernissimo · The Space Cinema Corciano · UCI Cinemas Perugia

Programmazioni sparse, formati diversi, nessun colpo d'occhio.

La soluzione

Un'unica app che raccoglie film, orari e cinema dai tre siti e li mostra in un posto solo.

Dati raccolti in automatico, non inseriti a mano.

Gratuita, senza registrazione, su web e smartphone.

Cosa fa, dal punto di vista di chi la usa

- Home con il cartellone del giorno e il filtro per data (i prossimi 7 giorni).
- Dettaglio del film con gli orari raggruppati per cinema e il link diretto all'acquisto.
- Ricerca per titolo, con debounce sulle chiamate al backend.
- Mappa dei tre cinema con marker e indirizzi, tap che apre in Google Maps.
- Stessa app e stessa codebase: gira su web e su smartphone (iOS/Android).

Requisiti funzionali

ID	Requisito	Priorità
RF-01	Lista dei cinema umbri (nome, città, indirizzo, sito)	Alta
RF-02	Programmazione di ogni cinema (film, orari, giorni)	Alta
RF-03	Cinema su mappa interattiva con marker cliccabili	Alta
RF-04	Filtro per data (oggi, domani, settimana)	Alta
RF-05	Filtro per comune / zona geografica	Media
RF-06	Scheda dettaglio film (genere, durata, poster, sinossi)	Media
RF-07	Link diretto al sito del cinema per l'acquisto	Alta
RF-08	Aggiornamento automatico via scraping schedulato (24h)	Alta
RF-09	Rispetto di robots.txt, niente pagine protette	Alta
RF-10	Ricerca per titolo	Media
RF-11	Avviso se i dati di un cinema sono vecchi di oltre 48h	Bassa

Requisiti non funzionali e priorità

Non funzionali (RNF-01 ... 07)

- Portabilità: iOS 16+ e Android 10+ da un unico codebase
- Aggiornamento dati entro 24h dalla fonte
- Privacy: nessun account, nessun tracciamento
- Compliance scraping: robots.txt, rate limit 1 req/s, run notturna
- Scalabilità: estendibile ad altre regioni senza cambi strutturali

Matrice importanza-difficoltà

Guida cosa entra nell'MVP e cosa si rimanda.

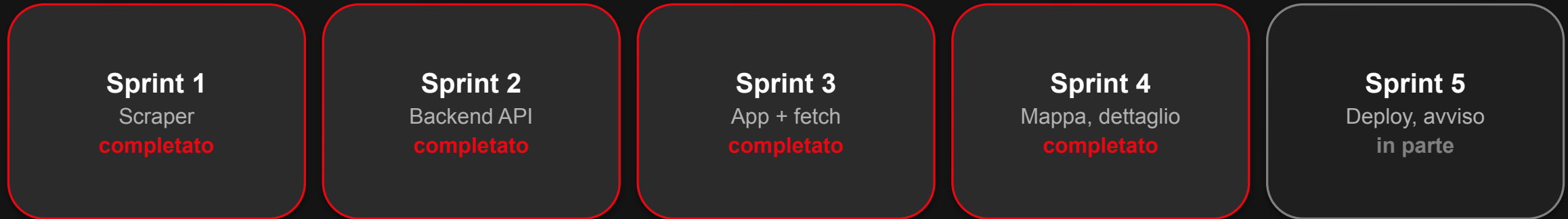
Nell'MVP: RF-01/02/03/04/06/07/09/10

Da pianificare con cura: RF-08

Da valutare: RF-05

Rilascio successivo: RF-11

Processo: Scrum per componente

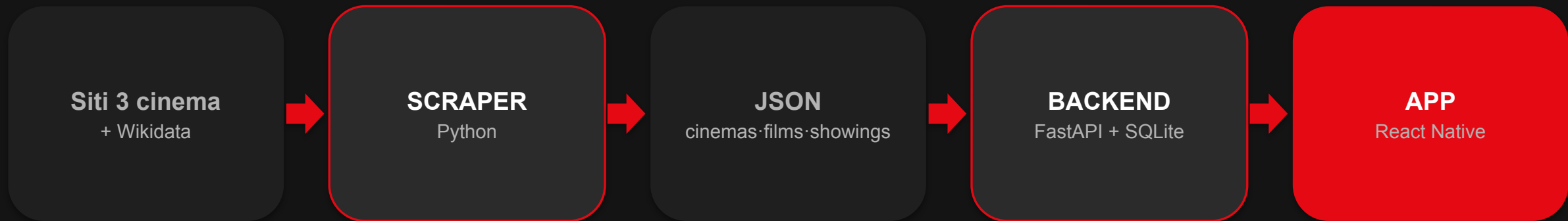


Ordine guidato dalle dipendenze

Prima lo scraper, per validare la fattibilità (i siti sono scrapabili?). Poi il backend, che sblocca il frontend con dati reali. Infine l'app. Ogni sprint consegna qualcosa di verificabile.

Rilasci incrementali: Alpha (solo API) → 1.0 MVP (oggi) → 1.1 (avviso dati e deploy).

Architettura pipeline a tre stadi



Contratti espliciti tra gli stadi: file JSON tra scraper e backend, API REST tra backend e app.

Ogni stadio è indipendente e si testa da solo. Lo scraper gira ogni notte (systemd timer); il backend importa i JSON con un seed idempotente; l'app consuma l'API.

Scraper: dai siti ai dati strutturati

Un connettore per cinema (pattern Strategy): stessa interfaccia, tecnica diversa per ogni fonte.

PostModernissimo

Sito Next.js — parsing del payload RSC + HTML (BeautifulSoup/lxml)

The Space

API REST OAuth2, con fallback CloakBrowser se l'API cambia

UCI

API Cloud Run non documentata, ricostruita dal traffico di rete

Poi, per ogni film raccolto:

Normalizza
titoli



Deduplica



Arricchisce
da Wikidata



Delta
tracking



Scrivi
3 JSON

Normalizzazione (accenti/punteggiatura via) per capire che "Dune – Parte 2" e "DUNE Parte 2" sono lo stesso film · **Wikidata** SPARQL con cache
· **delta tracking** tiene i dati vecchi marcandoli.

In produzione: **systemd timer alle 03:00** · **75 test**, lint ruff pulito.

Rischi e scraping etico

#	Rischio	Severità
R-01	Un sito cambia struttura, lo scraper si rompe	ALTO
R-03	Sito in JavaScript non scrapabile (SPA)	ALTO
R-02	robots.txt vieta il crawling	MEDIO
R-05	Scope creep (feature non pianificate)	MEDIO
R-04	Wikidata manca per film di nicchia	BASSO

Mitigazione chiave (R-01)

- Un connettore per cinema: si rompe solo quello
- Errori isolati, gli altri continuano
- Delta tracking: i dati vecchi restano
- 75 test fissano ogni parser

Scraping etico (RF-09 / RNF-04)

robots.txt rispettato · rate limit 1 req/s · run notturna · User-Agent con contatto reale · link al sito del cinema per l'acquisto: **portiamo traffico, non lo togliamo.**

Backend: API REST su FastAPI

routers/

ricevono HTTP, validano, scelgono lo status

services/

logica di business (es. "film di oggi")

repositories/

uniche classi che parlano col DB

models/

le tre tabelle SQLAlchemy

11 endpoint · **Swagger su /docs**

film /oggi · /settimana · /search · /{id}

cinema /cinema · /{slug} · /{slug}/showings

showings ?date=

admin /reimport · /dataset-info (token)

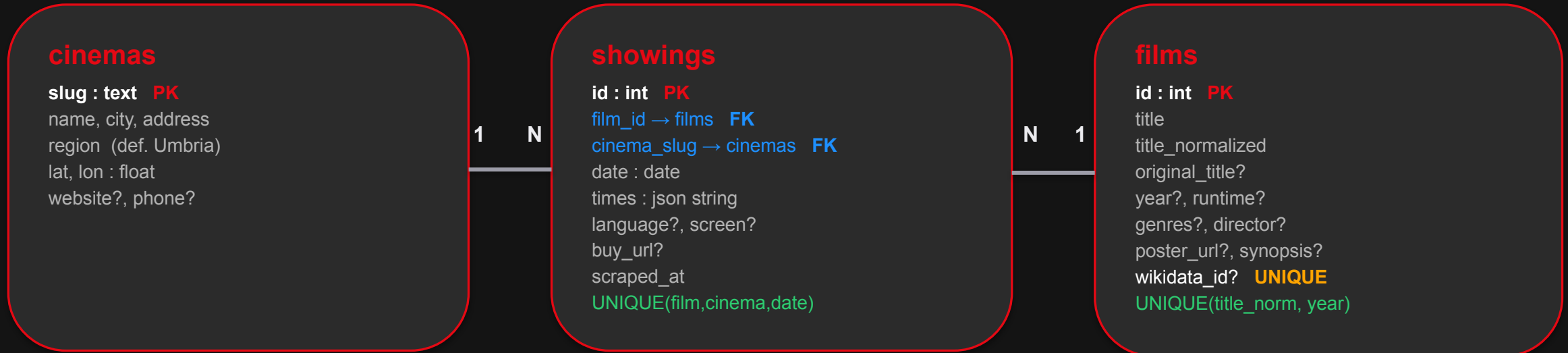
health /health

Tutti in sola lettura per l'app; i due **/admin** protetti da X-Admin-Token.

Ogni spettacolo annida il film e il cinema.

Stack: **FastAPI** · **SQLAlchemy 2.0** · **SQLite** · **Pydantic v2**. Seed **idempotente** dai JSON dello scraper (re-import senza duplicati). **26 test** (repository + endpoint con DB in-memory).

Schema del database



Legenda: **PK** chiave primaria **FK** → chiave esterna **UNIQUE** vincolo di unicità ? = campo opzionale

I due vincoli UNIQUE rendono il re-import notturno idempotente. **FK con ON DELETE CASCADE:** rimosso un film o un cinema, spariscono i suoi spettacoli. Indici su date, film_id, cinema_slug.

Pattern di progettazione e architetturali

Pattern	Dove	Perché
Strategy · GoF (comportamentale)	Connettori scraper	Stessa interfaccia scrape(), un'implementazione per cinema
Repository · architetturale (PoEAA)	backend/repositories	Isola l'accesso al DB dal resto del codice
Service layer · architetturale (PoEAA)	backend/services	Logica di business separata da HTTP e DB
DTO · architetturale (PoEAA)	schemas Pydantic	Contratto JSON stabile verso l'app
Dependency Injection · principio (IoC)	get_db in FastAPI	Testabilità: nei test si inietta un DB in-memory
Factory · creazionale (factory)	create_app()	Costruzione controllata di app e configurazione

Testing e qualità

101

test in tutto

75

scraper

26

backend

CI GitHub Actions

Due job (scraper + backend) a ogni push. Lint ruff pulito. Parser con fixture registrate; endpoint con DB in-memory.

Un bug reale, diagnosticato e corretto

The Space pubblica lo stesso film in più sale. Il vincolo `UNIQUE(film, cinema, data)` ne teneva una sola: "Odissea" mostrava 1 orario invece di 12.

Il seed ora unisce gli orari di tutte le sale prima di scrivere. Diagnosi → fix → test che blocca la regressione. Da 1 a 12 orari.

Decisioni di design difese

SQLite anche in produzione (D4)

Tre cinema, un paio di centinaia di spettacoli, sola lettura. Un DB server aggiungerebbe amministrazione senza benefici. Il layer repository isola la scelta: migrare a PostgreSQL cambia la stringa di connessione, non il codice.

Wikidata, non TMDb (D1)

Niente API key, niente limiti commerciali, licenza aperta, query SPARQL con cache locale.

Un connettore per cinema

Robustezza (un sito rotto non ferma gli altri) e flessibilità: l'API di UCI non è documentata e può cambiare.

Frontend: una sola codebase

Stack

React Native — 0.81 · React 19

Expo — SDK 54 (max per Expo Go)

Navigazione — React Navigation 7 (tab + stack)

Web — react-native-web, stesso codice nel browser

Mappa — Leaflet in WebView (nativo) / iframe (web)

Linguaggio — JavaScript (.js)

Navigazione (App.js)

All'avvio uno SplashScreen animato, poi:

SplashScreen,

└ **NavigationContainer**

└ **Bottom Tabs**

└ Films → stack (lista → dettaglio)

└ Cerca → stack (lista → dettaglio)

└ Località → mappa

Films e Cerca sono stack: il dettaglio si apre dentro la tab, con la barra sempre visibile.

Frontend: schermate e scelte tecniche

Films (Home)

carosello hero + barra 7 giorni + griglia locandine; filtro cinema e pull-to-refresh

Cerca

ricerca per titolo; le risposte obsolete vengono annullate (requestId)

Località

mappa Leaflet con i tre cinema e gli indirizzi; il tap apre in Google Maps

Dettaglio

poster, trama, trailer e orari raggruppati per cinema; si apre sulla data giusta

Scelte tecniche: locandina nitida su sfondo sfocato (i poster dei cinema sono a bassa risoluzione) · la sfocatura su web usa CSS filter perché blurRadius non funziona in react-native-web · date in ora locale per evitare il bug UTC di mezzanotte.

Demo dal vivo

L'app, gli orari veri raccolti stamattina, la pipeline end-to-end.

In sintesi

Fatto

- Pipeline completa end-to-end su dato reale
- App su web e smartphone, stessa codebase
- 101 test automatici + CI a ogni push
- Ricerca per titolo (era prevista per la 1.1)

Prossimi passi (Release 1.1)

- Avviso quando i dati di un cinema sono obsoleti
- Deploy in cloud (backend + web build)

Limiti onesti

UNIQUE(titolo, anno) non copre l'anno NULL (in SQL NULL $\neq$ NULL): lì la dedup è nel codice. L'API di UCI non è documentata. SQLite serializza le scritture — va bene perché scrive solo il seed notturno.

Grazie. Team RepCode